
Calclatorz

Arithmetic Expression Evaluator in C++

Test Cases for Parser Module

Version 1.1

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

Revision History

Date	Version	Description	Author
2026-05-04	1.0	Filling test case sections	Taryn
2026-05-06	1.1	Final polishing	Isaiah

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

Table of Contents

1. Purpose	4
2. Operator Precedence Test	4
3. Parentheses Handling Test	5
4. Nested Parentheses Test	6
5. Missing Operator Test	7
6. Unary Operator Handling	8
7. Consecutive Operators Test	9
8. Mismatch Parentheses Test	10
9. Invalid Character Test	11
10. Division by Zero Test	12
11. Exponential Input	13

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

Test Cases

1. Purpose

This *Test Cases* specification document for the Arithmetic Expression Evaluator in C++ defines the test cases for the `parser` module. The purpose of these tests is to verify correct conversion from infix to postfix notation, proper handling of operator precedence and parentheses, and detection of invalid expressions such as missing operators and syntax errors.

2. Operator Precedence Test

2.1 Test case identifier

PTC-001

2.2 Test item

Tests the parser's ability to correctly apply operator precedence rules (multiplication before addition).

2.3 Input specifications

Input expression: $2+3*4$

2.4 Output specifications

Expected Output: $2\ 3\ 4\ *\ +$

Actual Output: $2\ 3\ 4\ *\ +$

Result: Passed

2.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

3. Parentheses Handling Test

3.1 Test case identifier

PTC-002

3.2 Test item

Tests whether the parser correctly prioritizes expressions inside parentheses.

3.3 Input specifications

Input expression: $(2+3)*4$

3.4 Output specifications

Expected Output: $2\ 3 + 4 *$

Actual Output: $2\ 3 + 4 *$

Result: Passed

3.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

4. Nested Parentheses Test

4.1 Test case identifier

PTC-003

4.2 Test item

Verifies parser handles multiple nested parentheses correctly.

4.3 Input specifications

Input expression: $((2+3) * (4+1))$

4.4 Output specifications

Expected Output: 2 3 + 4 1 + *

Actual Output: 2 3 + 4 1 + *

Result: Passed

4.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

5. Missing Operator Test

5.1 Test case identifier

PTC-004

5.2 Test item

Tests detection of missing explicit operator between a number and parentheses

5.3 Input specifications

Input expression: 4(2+3)

5.4 Output specifications

Expected Output: Error: missing operator before '('

Actual Output: Error: Syntax error at position 2: missing operator before '('

Result: Passed

5.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

6. Unary Operator Handling

6.1 Test case identifier

PTC-005

6.2 Test item

Tests the parser's ability to handle unary operators at the beginning of an expression.

6.3 Input specifications

Input expression: +2+3

6.4 Output specifications

Expected Output: 2 + 3 +

Actual Output: Error: Syntax error at position 1: operator cannot appear at the beginning or end of the expression

Result: Failed

6.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

7. Consecutive Operators Test

7.1 Test case identifier

PTC-006

7.2 Test item

Verifies parser detects invalid sequences of operators.

7.3 Input specifications

Input expression: 2++3

7.4 Output specifications

Expected Output: Syntax Error

Actual Output: Error: Syntax error at position 3: unexpected operator placement

Result: Passed

7.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

8. Mismatch Parentheses Test

8.1 Test case identifier

PTC-007

8.2 Test item

Checks if parser detects unmatched parentheses

8.3 Input specifications

Input expression: (2+3

8.4 Output specifications

Expected Output: Mismatched parentheses error

Actual Output: Error: Mismatched parentheses: opening parenthesis does not have a matching closing parenthesis

Result: Passed

8.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

9. Invalid Character Test

9.1 Test case identifier

PTC-008

9.2 Test item

Test the tokenizer's ability to reject invalid characters.

9.3 Input specifications

Input expression: 7&3

9.4 Output specifications

Expected Output: Invalid character error

Actual Output: Error: Invalid character at position 2: unexpected character '&'

Result: Passed

9.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

10. Division by Zero Test

10.1 Test case identifier

PTC-009

10.2 Test item

Ensures evaluator handles division by zero safely

10.3 Input specifications

Input expression: 10/0

10.4 Output specifications

Expected Output: Division by zero error

Actual Output: Error: Division by zero at position 3: cannot divide by zero

Result: Passed

10.5 Environmental needs

Standard execution environment.

Arithmetic Expression Evaluator in C++	Version: 1.1
Test Cases for Parser	Date: 2026-05-06

11. Exponential Input

11.1 Test case identifier

PTC-010

11.2 Test item

Tests whether the system correctly recognizes and processes exponential (^) input.

11.3 Input specifications

Input expression: 2^3

11.4 Output specifications

Expected Output: 2^3

Actual Output: Error: Syntax error at position 3 (unexpected operator placement)

Result: Failed

11.5 Environmental needs

Requires tokenizer, parser, and evaluator modules to be running.